# CS6886 – Assignment 2

Madhav Bhardwaj
CS23B035

September 8, 2026

**Repository:** https://github.com/Madhav1729/cs6886-a2-mobilenetv2-compression
**W&B:** https://wandb.ai/cs23b035-iit-madras/cs6886-a2-mobilenetv2

## 1 Training Baseline

### 1.1 Data pipeline

I used CIFAR-10 with its standard split of 50,000 training and 10,000 test images. Table 1 lists the transforms.

| | Train | Test |
|---|---|---|
| 1 | `RandomCrop(32, padding=4, padding_mode='reflect')` | — |
| 2 | `RandomHorizontalFlip(p=0.5)` | — |
| 3 | `Resize(160, interpolation=BICUBIC)` | `Resize(160, BICUBIC)` |
| 4 | `ToTensor()` | `ToTensor()` |
| 5 | `Normalize(ImageNet mean/std)` | `Normalize(ImageNet mean/std)` |

Table 1: Input transforms. Normalization uses ImageNet statistics, $\mu = (0.485, 0.456, 0.406)$ and $\sigma = (0.229, 0.224, 0.225)$.

There are two main choices. First, we normalize with ImageNet statistics instead of CIFAR-10's own. The backbone starts from ImageNet weights, and those filters were trained on inputs scaled that way, so using CIFAR's statistics would shift the inputs away from what the pretrained features expect.

Second, the images are upsampled from $32 \times 32$ to $160 \times 160$. MobileNet-v2 reduces the spatial resolution significantly as it goes through the network. With a $32 \times 32$ input the feature maps become very small by the later layers – close to $1 \times 1$ – which does not leave much for the pretrained features to work with. Therefore resized the images to $160 \times 160$, which leaves a $5 \times 5$ feature map at the last stage. We picked 160 over the network's native 224 because in our experiments it gave similar accuracy for roughly half the compute per image.

The crop and flip run at $32 \times 32$ and the result is upsampled afterward. This should be equivalent to augmenting after the resize, but it is cheaper to compute, and the 4-pixel pad stays in CIFAR's own pixels rather than interpolated ones.

### 1.2 Model and training setup

I used the standard torchvision MobileNet-v2 architecture. Only the classifier is replaced with a new linear layer for 10 classes, so the rest of the ImageNet weights load directly. Table 2 gives the full configuration.

| Setting | Value |
|---|---|
| Architecture | `torchvision.models.mobilenet_v2` (unmodified) |
| Width multiplier | 1.0 |
| Dropout | 0.2 (stock, before the classifier) |
| BatchNorm | stock: momentum 0.1, $\varepsilon = 10^{-5}$, affine |
| Initialisation | `MobileNet_V2_Weights.IMAGENET1K_V1` |
| Classifier | `Linear(1280, 10)`, weights $\mathcal{N}(0, 0.01)$, zero bias |
| Parameters | 2,236,682 (8.53 MB in FP32) |
| Optimizer | SGD, momentum 0.9, Nesterov |
| Learning rate | 0.01 backbone, 0.1 classifier |
| Schedule | 1 epoch linear warmup, then cosine decay to zero |
| Weight decay | $4 \times 10^{-5}$, not applied to BatchNorm parameters or biases |
| Loss | cross-entropy, label smoothing 0.1 |
| Epochs | 20 |
| Batch size | 128 |
| Precision | mixed precision for training, FP32 for evaluation |
| Seed | 42 |

Table 2: Model configuration and training strategy.

The classifier is given ten times the backbone learning rate, since it starts from random weights while the backbone is already trained and should not need to move as much. We also use a one-epoch linear warmup to make the start of training more stable. Reasoning is that with the classifier randomly initialised, using the full learning rate immediately could produce large, noisy updates that affect the pretrained backbone before the classifier has settled down and warmup avoids this.

Weight decay is applied only to tensors with more than one dimension, which excludes BatchNorm parameters and all biases.

### 1.3 Results

The model reaches **96.34%** top-1 on the test set (Table 3), and Figure 1 shows the loss and accuracy curves.

| Metric | Value |
|---|---|
| Final-epoch test top-1 (reported) | **96.34**% |
| Best-epoch test top-1 (epoch 12) | 96.40% |
| Final-epoch train top-1 | 99.81% |
| Final-epoch test loss | 0.5916 |

Table 3: Baseline accuracy.

The last five epochs all fall between 96.31% and 96.38%, so picking the best epoch would mean selecting a model on the test set for a difference that is within noise.

### 1.4 Failure modes

Training accuracy ends at 99.81% against 96.34% on test, a gap of about three and a half. The test accuracy largely plateaus after around epoch 12, while the training loss keeps decreasing. This is a sign of some overfitting, and it suggests the network may have more capacity

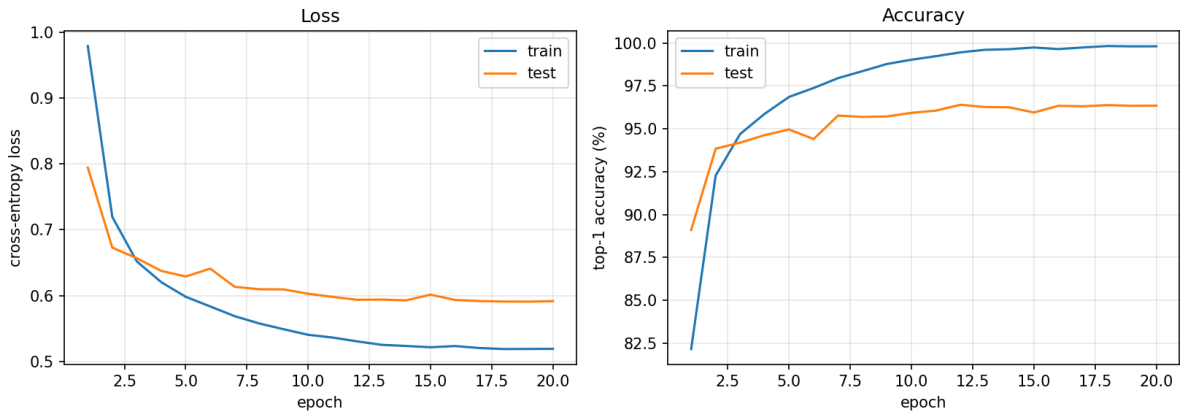


Figure 1: Loss (left) and top-1 accuracy (right) per epoch.

than CIFAR-10 actually needs. This was part of the motivation for exploring compression in Section 2.

The errors also vary considerably between classes. Table 4 shows `cat` at 91.2% and `dog` at 92.7%, both well below every other class. Looking at the confusion matrix, cats predicted as dogs (48 cases) and dogs predicted as cats (40 cases) together account for 88 of the 366 errors the model makes - almost a quarter of everything it gets wrong comes from this one pair. The next biggest source is trucks and automobiles being swapped (38 errors), then ships called airplanes (12). These are all pairs of visually similar classes, and at CIFAR's resolution the image detail that would normally help tell them apart is largely lost.

| | airplane | automobile | bird | cat | deer | dog | frog | horse | ship | truck |
|---|---|---|---|---|---|---|---|---|---|---|
| Top-1 (%) | 97.6 | 98.4 | 95.9 | **91.2** | 96.5 | **92.7** | 98.8 | 97.4 | 98.1 | 96.8 |

Table 4: Per-class test accuracy.

There is also a mismatch between what MobileNet-v2 was originally designed for and the images used here. The model was designed for $224 \times 224$ images, while CIFAR-10 images are only $32 \times 32$ and are resized to $160 \times 160$ before being given to the model. Resizing does not add any new information to the image it only makes the image larger so that the feature maps do not become too small inside the network. This may be one reason why the model has difficulty distinguishing similar classes such as cats and dogs, since the original images do not contain much fine detail.

## 2 Model Compression

Our compression method has three stages applied in sequence: pruning, then quantization, then Huffman coding. All three stages operate on a single configuration object, and quantization is what makes the method cover both weights and activations – pruning only affects weights, and Huffman coding is a lossless step on top of the quantized weights.

### 2.1 Design

**Pruning.** We use magnitude pruning: weights below a threshold are set to zero. The threshold is chosen globally across all target layers rather than per layer, because a global threshold lets layers with genuinely small weights give up more than layers where every weight matters. We checked this directly: pruning 50% of weights with a single global threshold left accuracy at

88.60% before any fine-tuning, while pruning the same 50% independently in each layer dropped accuracy to 19.40%. We also cap how much any one layer can lose (95% by default), since a global threshold can otherwise wipe out a small layer entirely – we saw exactly this happen to a depthwise layer at 85% global sparsity.

Choosing how much to prune turned out to matter more than how the pruning is allocated. We measured one-shot accuracy (before any fine-tuning) at several global sparsity levels: 50% left 88.60%, but 70% collapsed to 13.00%, and 80–85% were both around 9–9.25%, which is close to random guessing on ten classes. The drop between 50% and 70% is not gradual – the network is still mostly intact at 50% and effectively broken at 70%. We use 50% as our default sparsity for this reason: fine-tuning afterward only has to recover a moderate amount of accuracy, rather than rebuild a network that is close to guessing to begin with. This shows up clearly after fine-tuning too – combined with 4-bit quantization, 50% sparsity reached 93.23% accuracy at 10.74× compression, while 70% only reached 85.48% at 14.08×, a much larger accuracy cost for a comparatively small gain in compression.

**Weight quantization.** Weights are quantized to a signed integer range using a symmetric scale, $q = \text{round}(w/s)$, clipped to $[-2^{b-1}, 2^{b-1} - 1]$. The scale is computed per output channel rather than per tensor, since a single tensor-wide scale performs badly once channel magnitudes vary a lot, which they do in MobileNet-v2's depthwise layers (details in the next subsection). The scale itself can also be computed over groups of weights smaller than a full channel; we found that grouping in blocks of 128 gives a slightly better trade-off between accuracy and the overhead of storing extra scales than pure per-channel scaling, though the difference is small.

For picking the scale's clipping range, using the maximum absolute weight in a group is the simplest choice, but it is sensitive to outliers. We instead search over a small number of candidate clipping ratios and keep the one that minimises reconstruction error (an MSE-optimal scale). This gave a consistent accuracy improvement at low bit-widths – about 2 points at 4-bit weights over using the plain maximum.

**Activation quantization.** Activations are quantized per tensor with an asymmetric scale (a scale and a zero-point), since post-ReLU6 activations are non-negative while the linear bottleneck outputs before the residual add are signed. The quantization ranges are estimated by running a few calibration batches from the *training* set through the network and recording percentile-based ranges, then freezing them. Using the raw min/max instead of a percentile clip was noticeably worse: at 6-bit activations, min/max calibration gave 89.55% while clipping to the 99.9th percentile gave 95.85%. We calibrate on training data specifically so the reported test accuracy is not influenced by statistics taken from the test set.

One detail that took some debugging: MobileNet-v2's inverted residual blocks have two places where activations are not passed through a ReLU6 – the linear bottleneck projection and the residual addition. If quantizer hooks are only attached to ReLU6 modules, both of these are silently left in full precision, which would make the true activation compression ratio lower than what gets reported. We instead attach a quantizer to the output of every inverted residual block directly, in addition to every ReLU6, so both of these points are actually compressed.

**Huffman coding.** After quantization, the integer weight codes are not uniformly distributed – most weights end up small in magnitude, so low-valued codes (and, once pruning is applied, the code for zero) are far more common than the rest. We build a Huffman code from scratch over each tensor's code distribution and use it as the final storage format for the quantized weights. This is lossless: it does not change the accuracy at all, only the storage size. We measured that this consistently beats the nominal bit-width by 15–35% depending on how skewed the distribution is.

**Why pruning and Huffman coding combine cheaply.** Pruning usually requires extra storage to record which weights survived – a bitmap costs one bit per original position, and at 3–4 bit weights that index alone can cost as much as the weights themselves. We avoid this: a pruned weight is quantized to code zero like any other small weight, and since Huffman coding already assigns short codes to frequent symbols, a high proportion of zeros just makes that symbol cheaper to store. No separate index or bitmap is needed, and *no change to the size calculation was required to support pruning* – the masks only decide which weights become zero before quantization runs.

### 2.2 Configuration

All of the above is controlled by one dataclass:

| Parameter | Stage | Meaning |
|---|---|---|
| `weight_bits` | quantization | bits for the bulk (pointwise) weights |
| `sensitive_bits` | quantization | bits for depthwise/stem/classifier layers |
| `activation_bits` | quantization | bits for activation tensors |
| `group_size` | quantization | weights sharing one scale (0 = per channel) |
| `weight_clip` | quantization | scale selection rule (`max`, percentile, `mse`) |
| `act_clip` | quantization | activation calibration rule |
| `layer_bits` | quantization | optional per-layer bit override |
| `sparsity` | pruning | target fraction of weights pruned |
| `scope` | pruning | `global` or per-layer thresholding |
| `protect` | pruning | layers excluded from pruning |
| `max_layer_sparsity` | pruning | cap on any single layer's sparsity |

Table 5: Configuration parameters, grouped by which stage they control.

Sweeping any of these parameters and re-running the same three-stage pipeline is how we produce the different compression levels used in Section 3.

### 2.3 Applying it to MobileNet-v2

MobileNet-v2 is built almost entirely from two kinds of convolution: pointwise (1x1) convolutions, which mix channels, and depthwise convolutions, which operate on each channel independently. We treat these differently, along with the first (full) convolution and the final classifier, since together they make up only about 3.5% of all parameters but turned out to matter far more than their size suggests.

We checked this by measuring, layer by layer, how much accuracy is lost if only that one layer is quantized to 3 bits while every other layer stays at full precision. The results were striking: the network's largest layers, which are all pointwise convolutions with hundreds of thousands of parameters, lost almost nothing individually – some even showed a small positive change, which we take to be noise. In contrast, a single depthwise convolution with only 288 parameters caused a 50.80 point drop on its own. Table 6 lists a few examples.

Based on this, our default allocation is: pointwise convolutions (about 96.5% of parameters) are quantized aggressively, while depthwise convolutions, the first convolution, and the classifier are kept at 8 bits. We use the same split for pruning – only pointwise weights are pruned, and the same protected layers are excluded. Keeping these layers at higher precision costs very little storage, since they are such a small fraction of the model, but recovers a lot of accuracy. At 4-bit weights, quantizing everything uniformly gave 77.40% accuracy, while protecting these layers at 8 bits gave 91.80% – a 14 point gain for roughly 4% more storage.

One more detail affects which layers can be quantized safely: BatchNorm folding. Before quantizing, we fold every BatchNorm layer into the convolution before it, which is a standard,

| Layer | Kind | Parameters | Accuracy drop at 3-bit |
|---|---|---|---|
| `features.1.conv.0.0` | depthwise | 288 | 50.80 |
| `features.3.conv.1.0` | depthwise | 1,296 | 2.10 |
| `features.14.conv.2` | pointwise | 92,160 | 2.10 |
| `features.17.conv.2` | pointwise | 307,200 | 0.25 |
| `features.18.0` | pointwise | 409,600 | 0.00 |

Table 6: Per-layer sensitivity, measured by quantizing one layer at a time to 3 bits. Sensitivity does not track layer size – the largest layers here are the least sensitive.

lossless algebraic simplification at inference time and removes the BatchNorm parameters from storage entirely (34,112 parameters, about 133 KB). Folding multiplies each output channel by a different factor, though, and this widens the difference in scale between channels within a layer. We found this interacts badly with using a single scale per tensor: at 6-bit weights, per-tensor quantization dropped from 91.55% unfolded to 57.90% folded, while per-channel quantization gave 95.60% in both cases. So per-channel (or grouped) scales are not just a minor accuracy improvement here – they are what makes folding safe to do at all.

### 2.4 Storage overheads

Reporting compression as simply (bits per weight) $\times$ (number of weights) would understate the real model size, since the scales, the folded biases, and the Huffman code tables also need to be stored. Table 7 breaks this down for our W4/A8 configuration (4-bit pointwise weights, 8-bit protected layers, 8-bit activations, no pruning).

| Component | Size (KB) | Share |
|---|---|---|
| Huffman-coded weight payload | 1012.5 | 91.4% |
| Per-channel scales (FP16) | 33.3 | 3.0% |
| Folded biases (FP16) | 33.3 | 3.0% |
| Huffman code tables | 4.9 | 0.4% |
| **Total** | 1049.2 | 100% |

Table 7: Storage breakdown for W4/A8, per-channel scales. Overhead beyond the weight payload itself is about 6.5% of the total.

We also checked whether the scales themselves could be stored more cheaply than FP16. This turned out to matter a lot more than we expected. A scale multiplies every weight in its group, so an error in the scale is not the same kind of error as rounding an individual weight – it shifts a whole group of weights together rather than averaging out. We tested this directly: storing scales in FP16 gave 93.20% accuracy, BF16 gave 92.95%, and 8-bit floating point (e4m3) collapsed to 55.30%. FP16 scales cost very little (3% of the model here) and clearly should not be reduced further.

## 3 Compression Results

We ran the full pipeline at many combinations of weight bit-width, activation bit-width, which layers get the protected higher bit-width, and pruning sparsity, applying post-training quantization only (no fine-tuning) so that a full sweep is cheap – each configuration takes about 20 seconds to compress and evaluate. Table 8 shows a representative subset of an earlier 16-configuration run (weight bits $\times$ activation bits $\times$ sensitive bits, before we added pruning as a

swept axis).

| W bits | A bits | Sensitive bits | Accuracy | Model ratio | Size (MB) |
|---|---|---|---|---|---|
| 8 | 8 | 8 | 96.25% | 4.2× | 2.018 |
| 8 | 8 | 4 | 81.21% | 4.3× | 1.981 |
| 6 | 8 | 8 | 96.04% | 5.6× | 1.515 |
| 6 | 8 | 4 | 77.26% | 5.8× | 1.478 |
| 4 | 8 | 8 | 93.13% | 8.1× | 1.049 |
| 4 | 8 | 4 | 62.64% | 8.4× | 1.012 |
| 3 | 8 | 8 | 40.47% | 10.3× | 0.832 |
| 3 | 8 | 4 | 17.87% | 10.7× | 0.794 |

Table 8: Post-training accuracy (no fine-tuning) across weight bit-width, activation bit-width fixed at 8, and sensitive-layer bit-width. Dropping the protected layers to 4 bits costs 15–30 points of accuracy for less than 0.5× extra compression – this is the clearest evidence for keeping them at higher precision.

The most important pattern here is what happens to the *sensitive bits* column. Dropping the protected layers (depthwise, stem, classifier) from 8 to 4 bits is damaging at every weight width, and it barely improves compression: at 4-bit weights it costs 30 points of accuracy for a 0.3× gain in ratio. This is the same result described in Section 2, shown here as a sweep rather than a single comparison.

We then extended this sweep to also vary pruning sparsity, giving four axes in total: weight bits, activation bits, sensitive-layer bits, and sparsity, for $4 \times 2 \times 2 \times 2 = 32$ configurations. Figure 2 is the resulting Parallel Coordinates chart, with each line one configuration and its colour showing the resulting accuracy (dark blue is low, orange/yellow is high).

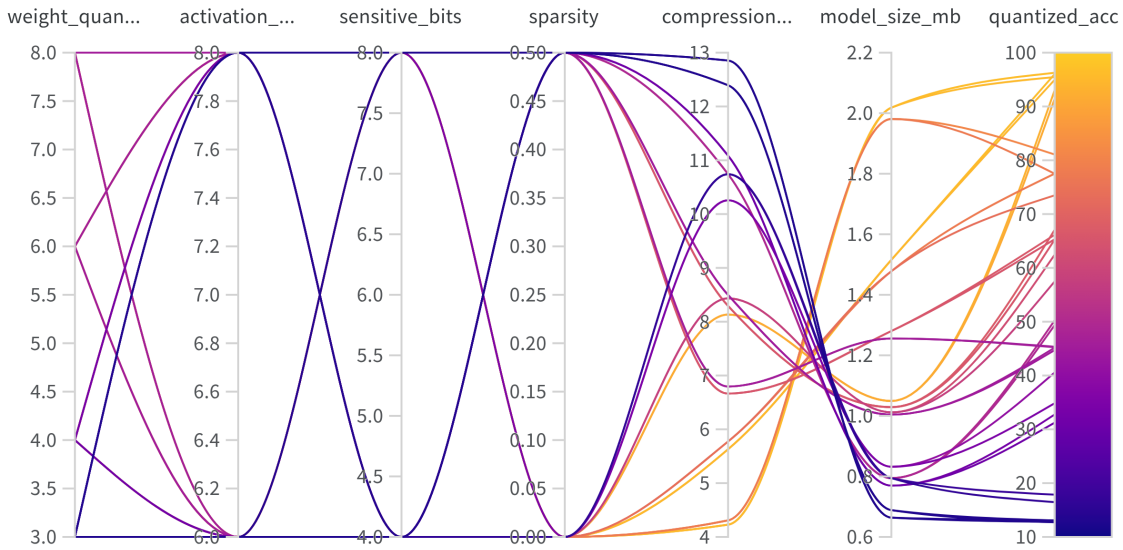


Figure 2: Parallel Coordinates chart over the 32-configuration sweep. Left to right: `weight_quant_bits`, `activation_quant_bits`, `sensitive_bits`, `sparsity`, `compression_ratio`, `model_size_mb`, `quantized_acc`. Colour encodes accuracy.

The clearest pattern in the chart is on the `sensitive_bits` axis: every line that dips to 4 there turns dark blue by the last column, meaning low accuracy, regardless of what the other three axes are doing. This matches Table 8 and the layer-by-layer results in Section 2 –

protecting the depthwise, stem, and classifier layers matters more than any other single choice in the sweep. Sparsity shows a milder version of the same pattern: lines with `sparsity`= 0.5 trend toward higher compression ratios but do not by themselves cause the sharp colour change that dropping `sensitive_bits` does. The orange, high accuracy lines all pass through `weight_quant_bits`= 8 or 6 with `sensitive_bits`= 8 and low sparsity, consistent with the rest of our results. All of the numbers in this section are post-training only, with no fine-tuning; fine-tuning recovers most of the accuracy lost at low bit-widths, which is what Section 4 reports.

# 4 Compression Analysis

Of the configurations we tried, we report the one that combines 4-bit quantization on the bulk of the weights with 50% magnitude pruning, since it gave the best accuracy for its compression ratio among everything we measured (Table 9).

| **Configuration** | **Accuracy** | **Model ratio** |
|---|---|---|
| W4/A8 (no pruning) | 94.97% | 8.13× |
| Sensitivity-guided mixed precision | 93.74% | 9.43× |
| W3/A8 (no pruning) | 93.10% | 9.91× |
| **W4/A8 + 50% pruning (chosen)** | **93.23%** | **10.74×** |

Table 9: Candidate configurations, all after fine-tuning. The chosen configuration reaches a higher compression ratio than uniform 3-bit weights while also being slightly more accurate.

The chosen configuration prunes 50% of the pointwise-convolution weights (global magnitude threshold, capped at 95% per layer), then quantizes the surviving weights to 4 bits (8 bits for the protected depthwise/stem/classifier layers, per-channel scales, MSE-optimal clipping), quantizes activations to 8 bits, and fine-tunes for 20 epochs with the pruning mask held fixed. Without fine-tuning this configuration only reaches 50.06%; fine-tuning brings it to 93.23%.

**(a) Weight compression ratio. 11.03×.** This measures the quantized-and-pruned weight storage against the original FP32 weight storage; it excludes the folded BatchNorm biases, which are reported separately in the size breakdown below.

**(b) Activation compression ratio. 4.0×** (32-bit down to 8-bit). We measure this as the ratio of bits per activation element, applied to every tensor a quantizer is attached to – every `ReLU6` output and every inverted-residual block output, which together cover all of the activation tensors that flow between compute stages in the network. Activations are never written to disk; they are recomputed on every forward pass, so this ratio is reported separately from model size rather than folded into it.

**(c) Accuracy at this compression ratio. 93.23%**, against an uncompressed baseline of 96.34% – a 3.11 point drop for 10.74× smaller storage. Before fine-tuning this configuration was at 50.06%; fine-tuning recovered 43.17 points of that.

**(d) Final model size. 0.795** MB, down from 8.53 MB in FP32. Table 10 gives the full breakdown, computed analytically rather than by measuring a serialized file, since a raw file size would be shaped by `torch.save`'s pickling and dtype choices rather than by the storage format we actually designed.

| Component | Size (KB) | Share |
|---|---|---|
| Huffman-coded weight payload | 742.5 | 91.3% |
| Per-channel scales (FP16) | 33.3 | 4.1% |
| Folded biases (FP16) | 33.3 | 4.1% |
| Huffman code tables | 4.5 | 0.5% |
| **Total** | 813.6 | 100% |

Table 10: Storage breakdown for the chosen configuration (W4/A8, 50% pruned).

# 5 Repository

https://github.com/Madhav1729/cs6886-a2-mobilenetv2-compression